Sample Project Documentation

Master technical documentation for AI WeatherWise API

Project: AI WeatherWise API   Track: AI-ML & GEN-AI Track

Prepared: 07 October 2026   Team ID: Not provided

Maximum Marks: 5

Team Members & Assigned Roles

Team Member	Role
Jai Bharath A	Team Lead & Lead Backend/API Architect
Mukesh Balaji	Generative AI & Prompt Engineering Engineer
Yadagiri Mani	Database & Data Engineering Specialist
Deepak G	Security, Testing & Reliability Engineer
Naveen	API Integration, Documentation & Demonstration Engineer

1. Executive Summary

AI WeatherWise API is a Gen-AI-enabled contextual weather backend. It combines live meteorological observations with Gemini-generated explanations and recommendations while retaining local fallback paths.

Key considerations

- Human-centered output
- Resilience by design
- Clear evaluator path

2. Background

Weather APIs are rich in numerical observations but poor at translating those values into decisions. WeatherWise addresses the interpretation gap.

Key considerations

- Interpretation gap
- Contextual recommendations
- API-first architecture

3. Problem Statement

Users need quick, understandable answers about weather impact on clothing, commuting and hydration rather than isolated measurements.

Key considerations

- Raw weather metrics
- Actionable advice
- Reliable availability

4. Objectives

Build a secure REST API; integrate live weather; add Gen-AI interpretation; persist favorite locations; provide resilient fallbacks; document and test the system.

Key considerations

- Secure access
- Live data
- Gen-AI intelligence

5. Scope

Authentication, profile, weather proxy, location CRUD, AI summary, AI recommendation, testing and deployment documentation are in scope.

Key considerations

- Defined endpoints
- Fallback paths
- Documentation

6. User Persona & Empathy

Primary users are students, commuters and everyday users who need actionable weather guidance.

Key considerations

- Student/commuter
- Fast decisions
- Low cognitive load

7. Ideation & Selection

WeatherWise was selected because it has a clear pain point, strong AI relevance, manageable implementation scope and real-world API integration.

Key considerations

- Feasible scope
- AI relevance
- Technical differentiation

8. Functional Requirements

The ten endpoints cover registration, login, profile, weather, four location operations, summary and recommendation.

Key considerations

- Functional behavior
- Validation
- Access control

9. Non-Functional Requirements

Security, resilience, maintainability, data integrity, performance and scalability are core quality requirements.

Key considerations

- Security
- Reliability
- Maintainability

10. Technology Stack

Node.js, Express.js, MongoDB Atlas/Mongoose, OpenWeatherMap, Gemini 2.5 Flash, bcryptjs, JWT, GitHub and Thunder Client/Postman.

Key considerations

- Open source runtime
- Managed data
- External AI/data APIs

11. Architecture

The system uses a layered MVC-style API with controllers, services, middleware and Mongoose models.

Key considerations

- Layer separation
- Reusable services
- Centralized middleware

12. Authentication & Security

Passwords are salted and hashed with bcrypt; JWT Bearer tokens protect profile and location resources; ownership checks isolate user data.

Key considerations

- Password protection
- Token verification
- Tenant isolation

13. Weather Intelligence

The weather service calls OpenWeatherMap and normalizes the response. A deterministic mock generator handles upstream failure.

Key considerations

- Live upstream data
- Normalization
- Mock fallback

14. Generative AI

Gemini 2.5 Flash receives controlled weather context and produces concise summaries and recommendations.

Key considerations

- Grounded prompts
- Response formatting
- Controlled context

15. Fallback Intelligence

Local rules can generate basic weather interpretation and lifestyle advice when Gemini is unavailable.

Key considerations

- Deterministic rules
- No cloud dependency
- Graceful degradation

16. Database Design

User records hold identity/authentication data; Location records reference the user and use a compound unique index on user, city and country.

Key considerations

- Schema validation
- Indexes
- Ownership references

17. API Endpoint Reference

Documented endpoints are listed below with methods, access and purpose. This section is intended to match the evaluator test collection.

Endpoint Reference

#	Method	Endpoint	Access	Purpose
1	POST	/api/auth/register	Public	Create user; bcrypt password hashing; unique email validation.
2	POST	/api/auth/login	Public	Validate credentials and issue a 30-day HMAC-SHA256 JWT.
3	GET	/api/auth/profile	Protected	Return the authenticated user's profile.
4	GET	/api/weather/:city	Public	Fetch real-time OpenWeatherMap data with deterministic mock fallback.
5	POST	/api/locations	Protected	Save a favorite city/location with ownership and uniqueness controls.
6	GET	/api/locations	Protected	List locations belonging only to the authenticated user.
7	PUT	/api/locations/:id	Protected	Update location nickname or personal notes.
8	DELETE	/api/locations/:id	Protected	Delete a location after ownership validation.
9	POST	/api/ai/weather-summary	Public	Generate a natural-language weather summary with Gemini 2.5 Flash.
10	POST	/api/ai/weather-recommendation	Public	Generate contextual outfit, commute and hydration recommendations.

18. Testing & Performance

Testing covers functional paths, invalid authentication, duplicate locations, external service failures and latency measurement.

Key considerations

- Positive tests
- Negative tests
- Latency evidence

19. Deployment & Operations

Secrets belong in environment variables; the service can be containerized and deployed to a cloud runtime.

Key considerations

- Environment variables
- Portable deployment
- Operational safety

20. Future Roadmap

Redis caching, severe-weather webhooks, route-based forecasts, multilingual voice AI, WebSockets and microservice separation are planned extensions.

Key considerations

- Caching
- Proactive alerts
- Multimodal AI

21. Team Contribution

The five-member team divides architecture, Gen-AI, data, security/testing and documentation/demo responsibilities.

Member	Role	Core Area
Jai Bharath A	Team Lead & Lead Backend/API Architect	Owns system architecture, Express.js MVC structure, API routing, server lifecycle, weather-service integration and final technical coordination.
Mukesh Balaji	Generative AI & Prompt Engineering Engineer	Owns Gemini 2.5 Flash integration, prompt design, weather-summary and recommendation logic, response formatting and Gen-AI fallback strategy.
Yadagiri Mani	Database & Data Engineering Specialist	Owns MongoDB Atlas/Mongoose schemas, User and Location models, CRUD design, compound indexing, data integrity and query optimization.
Deepak G	Security, Testing & Reliability Engineer	Owns bcrypt password protection, JWT security checks, endpoint testing, performance validation, failure scenarios and reliability safeguards.
Naveen	API Integration, Documentation & Demonstration Engineer	Owns API documentation, Thunder Client collections, test evidence, deployment/setup documentation, demo coordination and evaluator-facing communication.